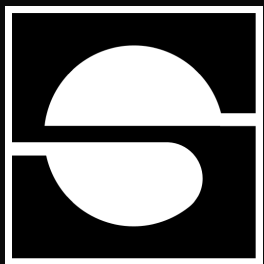




Security Assessment Final Report



Saturn Dollar

January 2026

Prepared for Saturn

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	3
Assessment Methodology.....	5
Threat and Security Overview.....	6
Findings Summary.....	8
Severity Matrix.....	8
Detailed Findings.....	9
Medium Severity Issues.....	11
M-01: Rounding Dust Loss in processRequests Causes Cumulative Value Leakage.....	11
M-02: redistributeLockedAmount() Can Be Sandwiched.....	13
Low Severity Issues.....	14
L-01: Underlying USDat Might Be Unbacked Due To Lag Between Offchain STRC Purchase And On Contract Burning Of USDat.....	14
L-02: Missing Handling for Withdrawals When Sufficient USDat Is Already Available.....	15
L-03: updateMinUsdatReceived Restriction Prevents Users from Adapting to Improved Execution Conditions.....	17
L-04: requestWithdraw Does Not Guarantee Asset Amount and Violates Withdraw Semantics.....	19
L-05: Users Can Bypass Deposit Fee By Donating Assets To The Vault.....	21
L-06: Mint And Deposit Results Deviate Due To Rounding.....	22
L-07: Deposit Fee Is Being Rounded Down.....	23
L-08: Processing Of A Non-existent tokenId Can Break Accounting And Freeze Further Withdrawals.....	25
Informational Severity Issues.....	27
I-01: Minor Inconsistencies and Suggestions.....	27
I-02: getStatus() Returns Misleading Value for Non-Existent Requests.....	29
I-03: IWithdrawalQueueERC721.addRequest Interface Does Not Match Implementation Return Value.....	30
I-04: maxDeposit and maxMint Do Not Reflect Paused State and Violate ERC4626 Requirements.....	32
I-05: depositWithMinShares and mintWithMaxAssets Bypass ERC4626 maxDeposit / maxMint Limits.....	33
Disclaimer.....	34
About Certora.....	34

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Saturn Dollar	https://github.com/saturn-organization/saturn-yield-dollar	c8c5a4c (initial commit) a55f288 (fix commit)	Solidity
	https://github.com/saturn-organization/saturn-dollar	bfc6c91	

Project Overview

This document describes the security review of **Saturn Dollar**. The work was undertaken from **January 16th, 2026** to **January 23rd, 2026**.

The following contract list is included in our scope:

saturn-organization/saturn-yield-dollar/src/*

saturn-organization/saturn-dollar/src/*

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Saturn is a DeFi protocol that issues a USD-denominated stablecoin (**USDat**) and a yield-bearing vault token (**sUSDat**) backed by STRC, a tokenized representation of MicroStrategy's preferred equity product. STRC trades on traditional markets with significant liquidity and pays recurring cash dividends, which the protocol aims to pass through to sUSDat holders.

USDat is a simple ERC20 stablecoin minted 1:1 against stable assets by KYC-verified users. Users can permissionlessly stake USDat to receive sUSDat, an ERC4626-style vault token representing a proportional claim on the protocol's NAV.

The sUSDat vault holds a combination of USDat and tokenizedSTRC, which mirrors the amount of STRC held by an off-chain custodian. The vault's `totalAssets()` is denominated in USD and reflects both on-chain USDat and the USD value of tokenized STRC priced via an oracle. This design enables on-chain tracking of NAV while off-chain buying and selling of STRC occurs asynchronously.

A privileged **PROCESSOR_ROLE** performs off-chain operations such as converting USDat to STRC, selling STRC to fulfill withdrawals, reinvesting dividends, and synchronizing execution results on-chain.

Withdrawals from sUSDat are asynchronous and handled through a dedicated **WithdrawalQueue** implemented as an ERC721 contract. Users submit withdrawal or redemption requests that escrow shares into the queue and specify a minimum USDat to receive. Requests are processed in batches based on FIFO-like ordering and market constraints, after which users can claim USDat once their requests are fulfilled.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

The primary security invariant in Saturn is that **the value represented by sUSDat shares must be conserved across conversions, yield realization, and withdrawals**, and that any shares burned during withdrawal processing correspond to value actually realized off-chain and reflected on-chain. Because settlement is delayed and execution occurs off-chain, correctness relies on enforcing these invariants through on-chain validation rather than atomic execution.

The most significant trust boundary is the `PROCESSOR_ROLE`, which supplies critical settlement parameters such as `totalStrcSold`, `totalUsdatReceived`, and `executionPrice`. While the protocol includes oracle checks, tolerance bounds, vesting constraints, and validation logic intended to reduce reliance on processor honesty, the processor remains capable of influencing economic outcomes. A central focus of the audit was determining whether on-chain checks sufficiently bind processor inputs to the actual liabilities being settled.

The audit considered attack vectors including:

- Processor under-settlement while still satisfying oracle and tolerance checks
- Share-to-asset accounting drift during batch withdrawal processing
- Rounding-induced value leakage and dust loss
- Withdrawal semantics that allow users to receive materially less than expected
- States where the vault holds USDat without sufficient STRC, blocking withdrawal execution
- ERC4626 non-compliance that could mislead integrators or automated systems
- Edge cases arising from pausing, partial conversions, and delayed execution

All core contracts were manually reviewed, including USDat, sUSDat, tokenizedSTRC, and the WithdrawalQueue. Invariants around share supply, NAV computation, and withdrawal settlement were analyzed across contract boundaries rather than in isolation.

The repository does not include test coverage for StakedUSDat or the associated withdrawal and processing modules. We recommend adding comprehensive tests covering all important scenarios to improve confidence in the protocol's correctness.



Overall, Saturn demonstrates a clear intent to constrain trust through on-chain validation. However, due to the delayed settlement model and reliance on privileged execution, **economic invariants and processor constraints remain the dominant security concern**, and correctness in these areas is critical to long-term protocol safety.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	–	–	–
Medium	2	2	1
Low	8	8	5
Informational	5	5	5
Total	15	15	11

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
M-01	Rounding Dust Loss in processRequests Causes Cumulative Value Leakage	Medium	Fixed
M-02	redistributeLockedAmount() can be sandwiched	Medium	Acknowledged
L-01	Underlying USDat might be unbacked due to lag between offchain STRC purchase and on contract burning of USDat	Low	Acknowledged
L-02	Missing Handling for Withdrawals When Sufficient USDat Is Already Available	Low	Acknowledged
L-03	updateMinUsdatReceived Restriction Prevents Users from Adapting to Improved Execution Conditions	Low	Fixed
L-04	requestWithdraw Does Not Guarantee Asset Amount and Violates Withdraw Semantics	Low	Fixed
L-05	Users can bypass deposit fee by donating assets to the vault	Low	Fixed
L-06	Mint and deposit results deviate due to rounding	Low	Acknowledged
L-07	Deposit fee is being rounded down	Low	Fixed

L-08	Processing of a non-existent tokenId can break accounting and freeze further withdrawals	Low	Fixed
I-01	Minor Inconsistencies and Suggestions	Informational	Fixed
I-02	getStatus() Returns Misleading Value for Non-Existent Requests	Informational	Fixed
I-03	IWithdrawalQueueERC721.addRequest Interface Does Not Match Implementation Return Value	Informational	Fixed
I-04	maxDeposit and maxMint Do Not Reflect Paused State and Violate ERC4626 Requirements	Informational	Fixed
I-05	depositWithMinShares and mintWithMaxAssets Bypass ERC4626 maxDeposit / maxMint Limits	Informational	Fixed

Medium Severity Issues

M-01: Rounding Dust Loss in processRequests Causes Cumulative Value Leakage

Severity: Medium	Impact: Low	Likelihood: High
Files: src/WithdrawalQueueERC721.sol	Status: Fixed	

Description:

In `processRequests`, each user's USDat payout is calculated pro-rata using `totalUsdatReceived`, `req.shares`, and `totalShares`, with the result rounded down.

```
uint256 usdatAmount = Math.mulDiv(totalUsdatReceived, req.shares,
totalShares, Math.Rounding.Floor);
totalUsdat += usdatAmount;
}

USDAT.mint(address(this), totalUsdat);
```

Because this rounding occurs for every request, small amounts are lost on each iteration. As a result, the accumulated `totalUsdat` distributed to users is very likely to be less than `totalUsdatReceived`. Currently, the contract mints only `totalUsdat` at the end of the function, meaning the rounding remainder (`totalUsdatReceived - totalUsdat`) is never minted and is effectively removed from the system.

Impact: This leads to permanent value loss on every batch execution. Over time, the unminted rounding dust accumulates into meaningful leakage.

Recommendations:

Mint the full `totalUsdatReceived` instead of `totalUsdat` so no value is lost. The rounding remainder should remain in the system and ideally be transferred to `stakedUSDat` (once sufficient value has

been accrued) so it is redistributed pro-rata to all existing users. Alternatively, the protocol can track this dust in a dedicated variable and mint it to **stakedUSDat** once a sufficient amount has accumulated.

Customer Response:

Fixed in [a0410a8](#).

Fix Review:

The issue is mostly addressed, but the transferred dust is still not reflected in the internal accounting. While the funds can be rescued by the admin, they would not be distributed back to existing users. Either approach is fine with us, we just wanted to explicitly confirm that this behavior is intentional and understood.

Customer Response:

Fixed in [2a40054](#).

Fix Review:

Fix looks good.

M-02: redistributeLockedAmount() Can Be Sandwiched

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/StakedUSDat.sol	Status: Acknowledged	

Description:

`redistributeLockedAmount()` burns all the shares of a blacklisted address i.e. `totalSupply()` drops and share value increases of the pre-existing users. Therefore a user can backrun a `redistributeLockedAmount()` call and deposit assets in `StakedUSDat` vault and when `redistributeLockedAmount()` executes it instantly increases the share value of the user and the user can then instantly queue a withdrawal at a profit setting his `minUsdatReceived` accordingly. This way the user can steal yield from the other users when the withdrawal requests are processed.

Recommendations:

The assets corresponding to the burnt shares should be accounted for in the `totalAssets` gradually i.e. as a function of time similar to the vesting logic.

Customer Response:

Acknowledge: 1 – The solution would introduce more complex code. 2 – This function is not likely to be called. 3 – A user who frontruns this function call, does not steal existing value from users but rather steals potentially new value that is not guaranteed to current holders.

Low Severity Issues

L-01: Underlying USDat Might Be Unbacked Due To Lag Between Offchain STRC Purchase And On Contract Burning Of USDat

Severity: Low	Impact: Low	Likelihood: Low
Files: src/StakedUSDat.sol	Status: Acknowledged	

Description:

`convertFromUsdat()` is called by the `PROCESSOR_ROLE` when STRC is purchased from the market and an equivalent amount of `TSTRC` is minted in the `StakedUSDat` vault and equivalent amount(based on STRC purchase price) of `USDat` is burnt in the vault. If there's a delay between the purchase of STRC from the market and the burning of `USDat` on chain then this would create a scenario where the `USDat` corresponding to the STRC purchased is unbacked in `StakedUSDat`.

Moreover, the assumption is that `USDat` is 1:1 pegged with US dollar therefore if any time in future `USDat` loses its peg temporarily then `totalAsset()` which is supposed to give a USD equivalent amount would give an incorrect result.

Recommendations:

It should be made sure that this delay between purchase of STRC from the market and the burning of `UDSat` on chain is negligible or as little as possible.

Customer Response:

Acknowledge: We will be restructuring our stablecoin and the flow will be as follows: 1. take a bridge loan to buy STRC that is a small percentage less than `USDat` in vault 2. Purchase STRC 3. update position and burn `USDat` in vault 4. off ramp stablecoins 5. pay back bridge loan

L-02: Missing Handling for Withdrawals When Sufficient USDat Is Already Available

Severity: Low	Impact: Low	Likelihood: Low
Files: src/WithdrawalQueueERC721. sol	Status: Acknowledged	

Description:

The current withdrawal flow assumes that USDat required for withdrawals will always be sourced by selling STRC off-chain and then minting USDat on-chain during `processRequests`. This tightly couples withdrawal processing to the STRC sale flow. If the contract holds sufficient USDat but little or no STRC, withdrawal processing becomes impossible without calling `convertFromUsdat`, even though no corresponding off-chain trade has been executed.

Although the protocol's intention is to keep most value in STRC, this scenario is still plausible. Operational decisions, extreme bad market conditions and the presence of functions like `convertToUsdat` indicate that the contract may hold USDat at times. In such cases, the current design provides no clear path to fulfill withdrawals without attempting an unnecessary or invalid STRC sale.

Impact: Withdrawals may be delayed or blocked even when the protocol has sufficient on-chain USDat to satisfy them. This introduces avoidable operational risk and makes withdrawal liveness dependent on executing off-chain STRC trades, even when it is not required.

Recommendations:

Introduce an alternative processing path for the processor role that allows queued withdrawal requests to be fulfilled directly using existing USDat balances, without selling STRC or calling `convertFromUsdat`.

Customer Response:



Acknowledge: This has larger implications on our protocol, we are planning to launch an upgrade in the future.

L-03: `updateMinUsdatReceived` Restriction Prevents Users from Adapting to Improved Execution Conditions

Severity: Low	Impact: Low	Likelihood: Low
Files: src/WithdrawalQueueERC721.sol	Status: Fixed	

Description:

The restriction in `updateMinUsdatReceived` that only allows decreasing the minimum amount appears to be intended to prevent griefing during `processWithdraw`, where users could otherwise increase their minimum and cause batch execution to revert. However, this restriction also undermines one of the main practical purposes of having an update function in the first place.

Withdrawal execution is asynchronous and may occur several days after the request is submitted. At the time of requesting, users cannot predict when execution will happen or what the execution price will be, as both vesting progress and STRC market prices may change in the interim. As a result, users are forced to set `minUsdatReceived` based solely on current conditions, even though the actual execution may occur under significantly more favorable conditions.

Because the function only allows `minUsdatReceived` to be decreased, users are unable to update their minimum to reflect improved execution expectations. For example, if a request is submitted when the expected payout is 100 USDat, but by the time execution approaches the expected payout increases to 120 USDat, the user remains locked into the original minimum and cannot express their updated expectation.

Recommendations:

Introduce a locking mechanism controlled by the admin. When the protocol is about to process withdrawals off-chain, the admin can enable a lock to prevent all users, or a selected subset of users, from calling `updateMinUsdatReceived`. Once the off-chain trade is finalized and on-chain execution is complete, the lock can be disabled again.

Customer Response:

Fixed in [64050e2](#).

Fix Review:

- The comment above `seizeRequests` should be updated from `Only works for Requested status` to `Only works for Requested and InProgress status`.
- The `RequestLocked` error does not appear to be used anywhere.

Customer Response:

Fixed in [7069c1c](#).

Fix Review:

Fix looks good.

L-04: requestWithdraw Does Not Guarantee Asset Amount and Violates Withdraw Semantics

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/StakedUSDat.sol	Status: Fixed	

Description:

In vault-style systems such as ERC4626, there is a clear semantic distinction between withdraw and redeem operations. Withdraw is asset-centric: the user specifies the amount of assets they want to receive, and the contract burns however many shares are required, subject to a `maxShares` slippage bound.

Redeem is share-centric: the user specifies the number of shares to burn, and the assets received may vary, subject to a `minAssets` bound.

This distinction allows users to explicitly choose whether they want certainty on the output assets or on the input shares.

In `requestWithdraw`, although the function asks the user to provide an `assets` amount, that amount is not guaranteed. It is only used to compute the number of shares escrowed at request time. The actual USDat received is determined later during off-chain execution and pro-rata settlement and can differ materially from the requested asset amount.

```
function requestWithdraw(uint256 assets, uint256 minUsdatReceived) external
whenNotPaused returns (uint256 shares) {
    require(assets <= maxWithdraw(msg.sender), ExcessiveRequestedAmount());

    shares = previewWithdraw(assets);

    _processWithdrawal(msg.sender, msg.sender, assets, shares,
minUsdatReceived);
}
```

Slippage protection is also applied on the wrong side. Instead of bounding the maximum number of shares that can be burned (which would guarantee the requested asset amount), the function uses `minUsdatReceived`, which only protects the output. As a result, the function behaves economically like a redeem operation, while being named and structured as a withdraw.

Impact:

Users relying on `requestWithdraw` to receive a fixed asset amount may pass `minUsdatReceived = 0`, assuming the asset amount is guaranteed. In practice, they may receive significantly less than expected depending on execution timing and market conditions, leading to unintended slippage and user loss.

Recommendations:

Given the current share-based, asynchronous processing model, a full ERC4626-style withdraw guarantee is not feasible without major changes. As a minimum improvement, when a request originates from `requestWithdraw`, the contract should automatically set `request.minUsdatReceived` to the user-provided `assets` amount inside `addRequest`, rather than relying on the user to supply a separate minimum.

Customer Response:

Fixed in [eb0e14e](#).

Fix Review:

The comment for the withdraw function is misleading now because we have removed `requestWithdraw`:

```
/// @notice ERC4626 withdraw is disabled – use requestWithdraw instead
```

Customer Response:

Fixed in [0c08982](#).

Fix Review:

Fix looks good.

L-05: Users Can Bypass Deposit Fee By Donating Assets To The Vault

Severity: Low	Impact: Low	Likelihood: Low
Files: src/StakedUSDat.sol	Status: Fixed	

Description:

The first depositor in the `StakedUSDat` vault can avoid paying the deposit fee by donating assets to the vault instead, consider the following scenario ->

1.) User wants to deposit 500e18 `USDat`, this deposit would incur a deposit fee(say 10%) which would be 50e18 and the user would be minted 450e18 shares and 50e18 is paid as deposit fee.

2.) Whereas if the user deposited say 10e18 assets and got 9e18 shares (paid 1e18 as deposit fee) and donated 490e18 assets directly to the vault then the user saves 49e18 assets, though as more users deposit and mint shares the user would loose out on this profit .This would be meaningful only during the initial deposits or if the user owns majority of the `StakedUSDat` vault.

Recommendations:

Track deposited assets internally rather than relying on `USDat.balanceOf()` else consider to acknowledge this behavior.

Customer Response:

Fixed in [a831e9d](#).

Fix Review:

Fix looks good.

L-06: Mint And Deposit Results Deviate Due To Rounding

Severity: Low	Impact: Low	Likelihood: Medium
Files: src/StakedUSDat.sol	Status: Acknowledged	

Description:

When a user calls `deposit()` the deposit logic calculates the fee from the asset provided by the user and mints shares corresponding to the remaining assets, whereas in `mint()` fee is accounted on the assets corresponding to the shares requested i.e. `Math.mulDiv(assets, BPS_DENOMINATOR, BPS_DENOMINATOR - depositFeeBps, Math.Rounding.Ceil);`

Consider the following scenario assuming 10% fee and precision to be 100 →

1. User `deposit(51 assets)` → `previewDeposit(51)` → $\text{fee} = 51 \cdot 10 / 100 = 5$, so user gets minted 46 shares and provides 51 assets
2. Whereas if he used `mint` → `mint(46 shares)` → `previewMint(46)` → $\text{assets} = 46$ → return $46 \cdot 100 / 90$ rounded up = 52 shares

Therefore, for the same amount of shares minted the user paid 1 wei of asset extra in `mint()` compared to `deposit()`.

Recommendations:

Consider acknowledging this behavior.

Customer Response:

Acknowledged.

L-07: Deposit Fee Is Being Rounded Down

Severity: Low	Impact: Low	Likelihood: Low
Files: src/StakedUSDat.sol	Status: Fixed	

Description:

The calculation of deposit fee is as follows →

```
function previewDeposit(uint256 assets) public view override returns (uint256) {  
    if (depositFeeBps == 0 || feeRecipient == address(0)) {  
        return super.previewDeposit(assets);  
    }  
  
    uint256 fee = Math.mulDiv(assets, depositFeeBps, BPS_DENOMINATOR);  
  
    return super.previewDeposit(assets - fee);  
}
```

The fee calculation is being rounded down against the protocol, fee should always be rounded in the favor of the protocol i.e. rounded up.

Recommendations:

Consider rounding up the fee calculation.

Customer Response:

Fixed in [822bd56](#).



Fix Review:

Fix looks good.

L-08: Processing Of A Non-existent tokenId Can Break Accounting And Freeze Further Withdrawals

Severity: Low	Impact: High	Likelihood: Low
Files: src/WithdrawalQueueERC721.sol	Status: Fixed	

Description:

If the `PROCESSOR_ROLE` provides a tokenId in the `tokenIds` array for which a withdrawal request does not exist then it would still pass every check in the `processRequests()` function and `pendingCount` will be decremented incorrectly →

```
for (uint256 i = 0; i < count; i++) {  
  
    Request storage req = requests[tokenIds[i]];  
  
    require(req.status == RequestStatus.Requested, AlreadyProcessed());  
  
    // Pro-rata: user gets their share of what was received  
  
    uint256 usdatAmount = Math.mulDiv(totalUsdatReceived, req.shares,  
totalShares, Math.Rounding.Floor);  
  
    // Validate against user's minimum  
  
    _validateAmount(usdatAmount, req.minUsdatReceived);  
}
```

```
    req.usdatOwed = usdatAmount;

    req.status = RequestStatus.Processed;

    totalUsdat += usdatAmount;

    emit WithdrawalProcessed(tokenIds[i], req.shares, usdatAmount);
}

require(totalUsdat <= totalUsdatReceived, ExecutionPriceMismatch());

pendingCount -= count;
```

This is because the function verifies the status to be requested and the default value of the `RequestStatus` enum is requested therefore it passes and `pendingCount` gets decremented. This could create a situation where `pendingCount` has been updated to 0 and there are still valid withdrawal requests to be processed but they cant be now since `pendingCount -= count` would underflow.

Recommendations:

Consider having a NULL value as the default value for the `RequestStatus` enum.

Customer Response:

Fixed in [806f31e](#).

Fix Review:

Fix looks good.

Informational Severity Issues

I-01: Minor Inconsistencies and Suggestions

Description:

1. Event naming inconsistency

- The `event toleranceUpdated(uint256 newToleranceBps)` uses a lowercase initial character, which is inconsistent with common Solidity and codebase naming conventions for events. While this has no functional impact, it reduces consistency across the contract events.

- **Recommendation:** Rename the event to `ToleranceUpdated(uint256 newToleranceBps)` and update all corresponding emissions.

2. Unused ERC20Permit inheritance in TokenizedSTRC

- `TokenizedSTRC` inherits from `ERC20Permit`, but there is no observable usage of permit-based approvals anywhere in the codebase. This adds unnecessary inheritance complexity and may confuse readers or integrators about intended functionality.

- **Recommendation:** Remove the `ERC20Permit` inheritance if permit functionality is not required, or explicitly integrate and document its intended use.

3. Incomplete economic validation in `_validateTotals`

- `processRequests` relies on `_validateTotals` to reduce trust assumptions on the `PROCESSOR` role, but the validation logic does not account for the value of the shares being redeemed. For example, users may have pending withdrawal requests totaling 1000 shares worth ~1100 `USDat`, yet the processor could supply parameters corresponding to only ~100 `USDat` (e.g., selling a small amount of `STRC` at a valid oracle price). All current validations would pass, but users would collectively receive far less value than the shares being burned.

- **Recommendation:** Extend `_validateTotals` to derive the expected `USDat` (or `STRC`-equivalent) value from `totalShares` being processed and verify that `totalUsdatReceived` falls within an acceptable tolerance of that value, ensuring that on-chain checks reflect the economic settlement of the withdrawals.

Recommendations:



See in the Description.

Customer Response:

Fixed in [891228b](#).

Fix Review:

Fix looks good.

I-02: `getStatus()` Returns Misleading Value for Non-Existent Requests

Files:

src/WithdrawalQueueERC721.sol

Description:

`getStatus(uint256 tokenId)` directly returns `requests[tokenId].status` without validating that the request exists. Since uninitialized storage entries default to zero, querying a non-existent `tokenId` will return `RequestStatus.Requested` (the zero enum value), even though no such request was ever created.

```
function getStatus(uint256 tokenId) external view returns (RequestStatus) {  
    return requests[tokenId].status;  
}
```

This can mislead off-chain indexers, frontends, or integrators into treating non-existent withdrawal requests as valid and pending.

Recommendations:

Consider adding an explicit `NonExistent` value to the `RequestStatus` enum to distinguish uninitialized requests from valid ones. Alternatively, validate `tokenId` against a known range (for example, `tokenId < nextTokenId`) and revert or handle the call accordingly when the request does not exist.

Customer Response:

Fixed in [806f31e](#).

Fix Review:

Fix looks good.

I-03: IWithdrawalQueueERC721.addRequest Interface Does Not Match Implementation Return Value

Files:

src/interfaces/IWithdrawalQueueERC721.sol

Description:

In the `IWithdrawalQueueERC721` interface, `addRequest` is declared as:

```
function addRequest(address owner, uint256 shares, uint256 minUsdatReceived)
external;
```

However, the actual implementation of `addRequest` returns a `tokenId`:

```
function addRequest(address user, uint256 shares, uint256 minUsdatReceived)
    external
    returns (uint256 tokenId)
```

This creates an inconsistency between the interface and the actual contract behavior. Any contract or external system interacting with the withdrawal queue through the interface cannot access the returned `tokenId`, despite it being a meaningful output of the function.

Recommendations:

Update the `IWithdrawalQueueERC721` interface to include the return value:

```
function addRequest(address owner, uint256 shares, uint256 minUsdatReceived)
external returns (uint256 tokenId);
```

Customer Response:

Fixed in [ceOccle](#).

Fix Review:

While the fix is correct, it would be better to also return the request ID to users during `requestRedeem`.



Customer Response:

Fixed in [c7864e6](#).

Fix Review:

Fix looks good.

I-04: maxDeposit and maxMint Do Not Reflect Paused State and Violate ERC4626 Requirements

Files:

StakedUSDat

Description:

As per the ERC4626 [specification](#), `maxDeposit` and `maxMint` **must factor in both global and user-specific limits**. The spec explicitly states that if deposits or mints are entirely disabled, even temporarily, these functions **must return 0** to accurately reflect that no new deposits or mints are allowed.

MUST factor in both global and user-specific limits, like if mints are entirely disabled (even temporarily) it MUST return 0.

In `StakedUSDat`, the contract relies on the default ERC4626 implementations of `maxDeposit` and `maxMint`, which return `type(uint256).max`. However, the contract is pausable, and when it is paused, deposits and mints are effectively disabled. Despite this, `maxDeposit` and `maxMint` continue to return unlimited values, which is inconsistent with ERC4626 requirements.

This results in ERC4626 non-compliance and can mislead integrators, frontends, and automated systems that rely on `maxDeposit` and `maxMint` to determine whether deposits or mints are currently permitted. During paused states, these systems may attempt operations that are guaranteed to revert

Recommendations:

Override `maxDeposit` and `maxMint` to return 0 whenever the contract is paused, and otherwise fall back to the existing logic.

Customer Response:

Fixed in [1b5882f](#).

Fix Review:

Fix looks good.

I-05: depositWithMinShares and mintWithMaxAssets Bypass ERC4626 maxDeposit / maxMint Limits

Files:

src/StakedUSDat.sol

Description:

ERC4626 enforces `maxDeposit` and `maxMint` limits to ensure deposits and mints respect protocol-level constraints such as caps, pauses, liquidity conditions, or risk controls. The standard `deposit` and `mint` functions correctly enforce these limits.

However, the newly added `depositWithMinShares` and `mintWithMaxAssets` functions in `StakedUSDat` do not perform any checks against `maxDeposit` or `maxMint`. As a result, users can bypass the intended ERC4626 limits by using these alternative entry points.

While the contract currently uses default implementations that return `type(uint256).max`, this assumption may not hold in the future if protocol-level limits are introduced.

Recommendations:

Ensure that `depositWithMinShares` enforces `assets <= maxDeposit(receiver)` and that `mintWithMaxAssets` enforces `shares <= maxMint(receiver)`, even if the current implementation returns `uint256.max`. This future-proofs the contract and keeps all deposit and mint paths consistent with ERC4626 guarantees and protocol risk controls.

Customer Response:

Fixed in [6f75918](#).

Fix Review:

This is properly fixed. Just FYI that standard ERC4626 errors like `ERC4626ExceededMaxDeposit` and `ERC4626ExceededMaxMint` could be used instead.

Customer Response:

Fixed in [2389971](#).

Fix Review: Fix looks good.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.